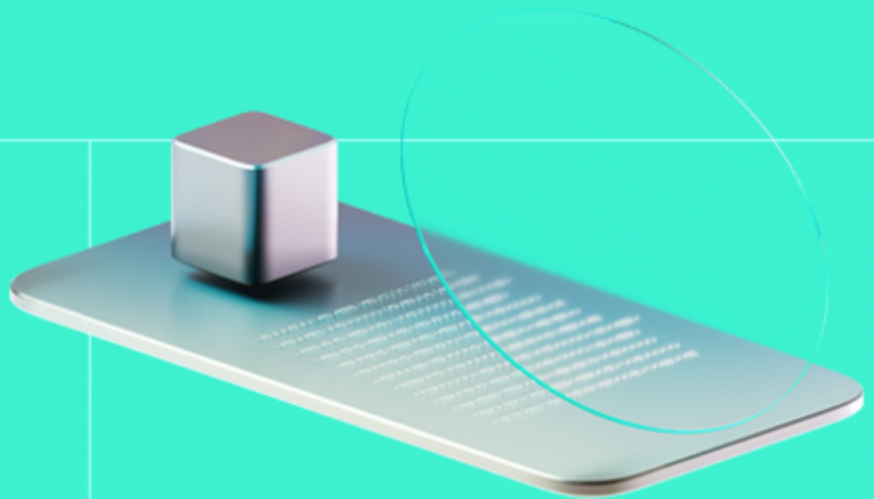




Smart Contract Code Review And Security Analysis Report

Customer: Adprima

Date: 22/01/2026



We express our gratitude to the AdPriva team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

The **Anchor** contract provides a public, verifiable log of AdPriva proof batches, recording batch hashes and Merkle roots on-chain for auditability while relying on off-chain filtering to identify authorized submissions.

Document

Name	Smart Contract Code Review and Security Analysis Report for AdPriva
Audited By	Kornel Światłowski
Approved By	Turgay Arda Usman
Website	-
Changelog	14/01/2026 - Preliminary Report
	22/01/2026 - Final Report
Platform	Base
Language	Solidity
Tags	Event Emitter
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Initial Repository	https://github.com/AdPriva/contracts
Initial Commit	411fa915898be15c59209d0a4704de8c305a8061
Final Repository	https://github.com/AdPriva/adpriva-contracts
Final Commit	9a4dccfdda19a15170c35fb8d8684be0e3d9f10d

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

6	1	5	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	0
Low	0

Vulnerability	Severity Status	
F-2026-14772 - Floating Pragma	Info	Fixed
F-2026-14773 - Custom Errors Can Be Used for Gas Efficiency	Info	Accepted
F-2026-14774 - Repeated Array Length Calculation Increases Gas Consumption	Info	Accepted
F-2026-14775 - Redundant Timestamp Emission Increases Transaction Cost	Info	Accepted
F-2026-14776 - Public Anchor Functions Allow Event Spamming	Info	Accepted
F-2026-14777 - Unnecessary Default Value Assignment Of Counter Variable	Info	Accepted

Documentation quality

- Functional requirements are detailed.
 - Project overview is detailed
 - All roles in the system are described.
 - All interactions are described.
- Technical description is detailed.
 - Run instructions are provided.
 - Technical specification is provided.
 - NatSpec is sufficient.

Code quality

- The development environment is configured.
- Insufficient Gas modeling.
- Code follows Solidity Style Guide.
- NatSpec covers the code and is detailed.

Test coverage

Code coverage of the project is **100%** (branch coverage).

- Deployment and basic user interactions are covered with tests.
- Negative cases coverage is present.

Table of Contents

System Overview	6
Privileged Roles	6
Potential Risks	7
Findings	8
Vulnerability Details	8
Disclaimers	20
Appendix 1. Definitions	21
Severities	21
Potential Risks	21
Appendix 2. Scope	22
Appendix 3. Additional Valuables	23

System Overview

The **Anchor** contract provides a public, verifiable log of AdPriva proof batches, recording batch hashes and Merkle roots on-chain for auditability while relying on off-chain filtering to identify authorized submissions.

Privileged roles

The project does not contain privileged roles.

Potential Risks

Findings

Vulnerability Details

F-2026-14772 - Floating Pragma - Info

Description: In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.20`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations (`^0.8.20`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

Assets:

- contracts/Anchor.sol [<https://github.com/AdPriva/contracts>]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to lock the pragma version to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates.

Resolution:

The issue was fixed in commit `9a4dccf`. The floating pragma was locked to version `0.8.20`.

F-2026-14773 - Custom Errors Can Be Used for Gas Efficiency - Info

Description:

Starting from Solidity version `0.8.4`, the language introduced a feature known as "custom errors". These custom errors provide a way for developers to define more descriptive and semantically meaningful error conditions without relying on string messages. Prior to this version, developers often used the `require` statement with string error messages to handle specific conditions or validations. However, every unique string used as a revert reason consumes gas, making transactions more expensive.

Custom errors, on the other hand, are identified by their name and the types of their parameters only, and they do not have the overhead of string storage. This means that, when using custom errors instead of `require` statements with string messages, the gas consumption can be significantly reduced, leading to more gas-efficient contracts.

Assets:

- `contracts/Anchor.sol` [<https://github.com/AdPriva/contracts>]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

It is recommended to use custom errors instead of revert strings to reduce increased Gas usage, especially during contract deployment. Custom errors can be defined using the `error` keyword and can include dynamic information.

Current approach:

```
require(batchIdHash != bytes32(0), "Invalid batchIdHash");
```

Usage of custom errors:

```
// Solidity version 0.8.26 or higher
// custom error declaration
error InvalidBatchIdHash();

require(batchIdHash != bytes32(0), InvalidBatchIdHash());
```

Resolution:

The issue was accepted by the Adpriva team.

[F-2026-14774](#) - Repeated Array Length Calculation Increases Gas Consumption - Info

Description:

The `anchorBatch()` function is responsible for batching multiple proof batches into a single transaction. The function iterates over the `batchIdHashes` array provided as a function argument and performs validation and processing for each element.

```
function anchorBatch(  
    bytes32[] calldata batchIdHashes,  
    bytes32[] calldata merkleRoots,  
    string calldata chainTag,  
    string calldata note  
) external {  
    require(batchIdHashes.length > 0, "Empty array");  
    require(batchIdHashes.length <= MAX_BATCH_SIZE, "Batch too large");  
    require(  
        batchIdHashes.length == merkleRoots.length,  
        "Array length mismatch"  
    );  
    require(bytes(chainTag).length > 0, "Empty chainTag");  
    require(bytes(chainTag).length <= MAX_CHAIN_TAG_LENGTH, "chainTag too long");  
    require(bytes(note).length <= MAX_NOTE_LENGTH, "note too long");  
  
    for (uint256 i = 0; i < batchIdHashes.length; i++) {  
        require(batchIdHashes[i] != bytes32(0), "Invalid batchIdHash");  
        require(merkleRoots[i] != bytes32(0), "Invalid merkleRoot");  
  
        emit Anchored(  
            batchIdHashes[i],  
            merkleRoots[i],  
            msg.sender,  
            block.timestamp,  
            chainTag,  
            note  
        );  
        unchecked { ++i; }  
    }  
}
```

During function execution, the length of the `batchIdHashes` array is repeatedly evaluated multiple times. Although retrieving an array length is a constant-time operation, repeated access introduces unnecessary gas

overhead during execution, especially when the array contains a large number of elements.

This pattern results in avoidable gas consumption and reduces overall execution efficiency. The array length remains constant throughout the function execution and can be safely cached once and reused during iteration.

Assets:

- contracts/Anchor.sol [<https://github.com/AdPriva/contracts>]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations**Remediation:**

It is recommended to cache the length of the `batchIdHashes` array in a local variable at the beginning of the function and reuse it throughout the function flow, instead of recalculating the array length multiple times during execution.

```
function anchorBatch(  
    bytes32[] calldata batchIdHashes,  
    bytes32[] calldata merkleRoots,  
    string calldata chainTag,  
    string calldata note  
) external {  
    uint256 batchIdHashesLength = batchIdHashes.length;  
    require(batchIdHashesLength > 0, "Empty array");  
    require(batchIdHashesLength <= MAX_BATCH_SIZE, "Batch too large");  
    require(  
        batchIdHashesLength == merkleRoots.length,  
        "Array length mismatch"  
    );  
    require(bytes(chainTag).length > 0, "Empty chainTag");  
    require(bytes(chainTag).length <= MAX_CHAIN_TAG_LENGTH, "chainTag too  
long");  
}
```

```
require(bytes(note).length <= MAX_NOTE_LENGTH, "note too long");

for (uint256 i = 0; i < batchIdHashesLength;) {
    // ...
}
}
```

Resolution:

The issue was accepted by the Adpriva team.

F-2026-14775 - Redundant Timestamp Emission Increases

Transaction Cost - Info

Description:

The `Anchor` contract contains two functions that emit events based on provided function arguments. These events are subsequently captured and filtered by AdPriva's backend.

The `Anchored` event includes a `timestamp` parameter, which records the time when the event is emitted. This parameter is redundant because the transaction timestamp can be obtained directly from the blockchain. Including this parameter increases transaction gas cost without adding unique value.

```
event Anchored(  
    bytes32 indexed batchIdHash,  
    bytes32 indexed merkleRoot,  
    address indexed submitter,  
    uint256 timestamp,  
    string chainTag,  
    string note  
);
```

The current design results in avoidable gas expenditure and reduces efficiency in event emission.

Assets:

- `contracts/Anchor.sol` [<https://github.com/AdPriva/contracts>]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

It is recommended to remove the `timestamp` parameter from the `Anchored` event, relying on the blockchain-provided block timestamp for temporal information. This will reduce transaction gas costs while maintaining all required event data.

Resolution:

The issue was accepted by the Adpriva team.

F-2026-14776 - Public Anchor Functions Allow Event Spamming - Info

Description:

According to the NatSpec documentation:

** Access Control: Anyone can call anchor() - this is intentional.
The contract serves as a neutral public anchoring log. AdPriva's backend filters events by the authorized submitter address, ignoring unauthorized anchors.*

The `anchor()` and `anchorBatch()` functions are publicly callable. This design allows any actor to submit a large number of events, which can overload the backend filtering system. Such event flooding can reduce efficiency and delay the processing of valid anchoring events.

```
function anchor(  
    bytes32 batchIdHash,  
    bytes32 merkleRoot,  
    string calldata chainTag,  
    string calldata note  
    ) external returns (bytes32) {...}  
  
function anchorBatch(  
    bytes32[] calldata batchIdHashes,  
    bytes32[] calldata merkleRoots,  
    string calldata chainTag,  
    string calldata note  
    ) external {...}
```

The current implementation lacks restrictions on who can emit anchor events, leading to potential griefing attacks.

Assets:

- `contracts/Anchor.sol` [<https://github.com/AdPriva/contracts>]

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity:

Info

Recommendations**Remediation:**

It is recommended to restrict access to the `anchor()` and `anchorBatch()` functions to permitted addresses. This can be implemented using a storage variable for a single permitted address, a mapping for multiple addresses, or by integrating OpenZeppelin's `AccessControl` library for structured access management. This approach prevents unauthorized event spamming while maintaining the intended public logging functionality.

Resolution:

The issue was accepted by the Adpriva team.

F-2026-14777 - Unnecessary Default Value Assignment Of Counter Variable - Info

Description: The `anchorBatch()` function contains a loop where the counter `i` is explicitly assigned a value of `0`. Since the counter is declared as `uint256`, its default value is already `0`. This assignment is redundant and increases code verbosity without affecting functionality.

```
for (uint256 i = 0; i < batchIdHashes.length;) {}
```

Assets:

- `contracts/Anchor.sol` [<https://github.com/AdPriva/contracts>]

Status: Accepted

Classification

Impact Rate: 1/5
Likelihood Rate: 5/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation: It is recommended to remove the redundant initialization of the loop counter `i` in the `anchorBatch()` function, relying on the default `uint256` value of `0`. This reduces unnecessary code and slightly optimizes gas usage.

Resolution: The issue was accepted by the Adpriva team.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Initial Repository	https://github.com/AdPriva/contracts
Initial Commit	411fa915898be15c59209d0a4704de8c305a8061
Final Repository	https://github.com/AdPriva/adpriva-contracts
Final Commit	9a4dccfdda19a15170c35fb8d8684be0e3d9f10d
Whitepaper	N/A
Requirements	NatSpec
Technical Requirements	NatSpec

Asset	Type
contracts/Anchor.sol [https://github.com/AdPriva/contracts]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.